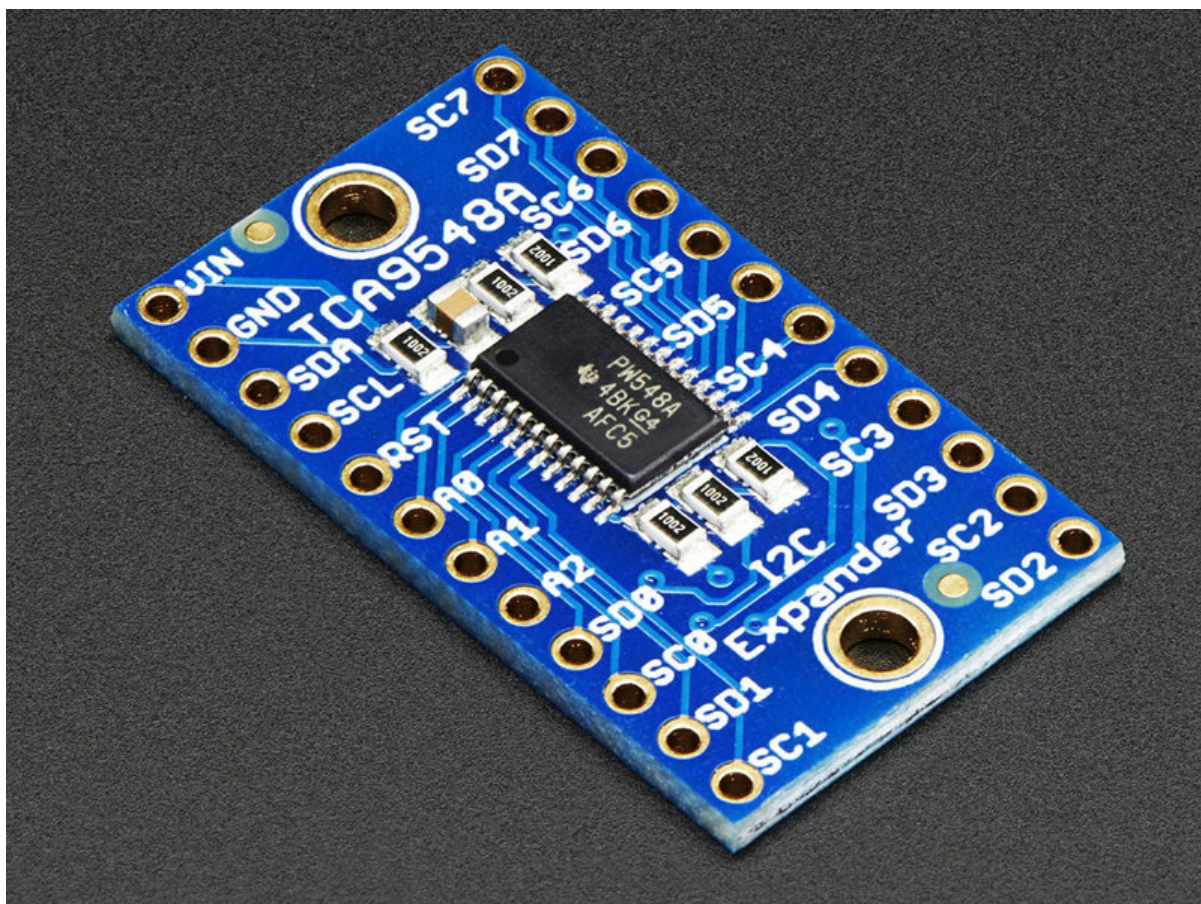




Adafruit TCA9548A 1-to-8 I2C Multiplexer Breakout

Created by lady ada



<https://learn.adafruit.com/adafruit-tca9548a-1-to-8-i2c-multiplexer-breakout>

Last updated on 2026-08-20 03:31:18 PM UTC

Table of Contents

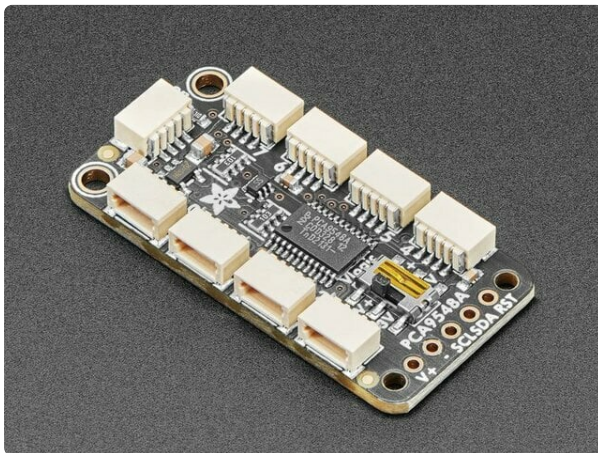
Overview	3
Pinouts	6
• Power Pins: • I2C Control-Side pins: • I2C Address Selection • I2C Multiplexed-Side pins:	
Assembly	8
• Prepare the header strip: • Add the breakout board: • And Solder!	
Arduino Wiring & Test	11
• Example Multiplexing • Multiple Multiplexers	
CircuitPython & Python	17
• CircuitPython Microcontroller Wiring • Python Computer Wiring • Python Installation of TCA9548A Library • CircuitPython Usage • Python Usage • Simple Test Example Code • Multi-Sensor Example Code	
Downloads	24
• Datasheets • Schematic • Fabrication Print	

Overview



Text emphasized
with a green
exclamation:

Also see the guide on the Adafruit PCA9548 which is
TCA9548A compatible and has Stemma QT ports for
I2C.

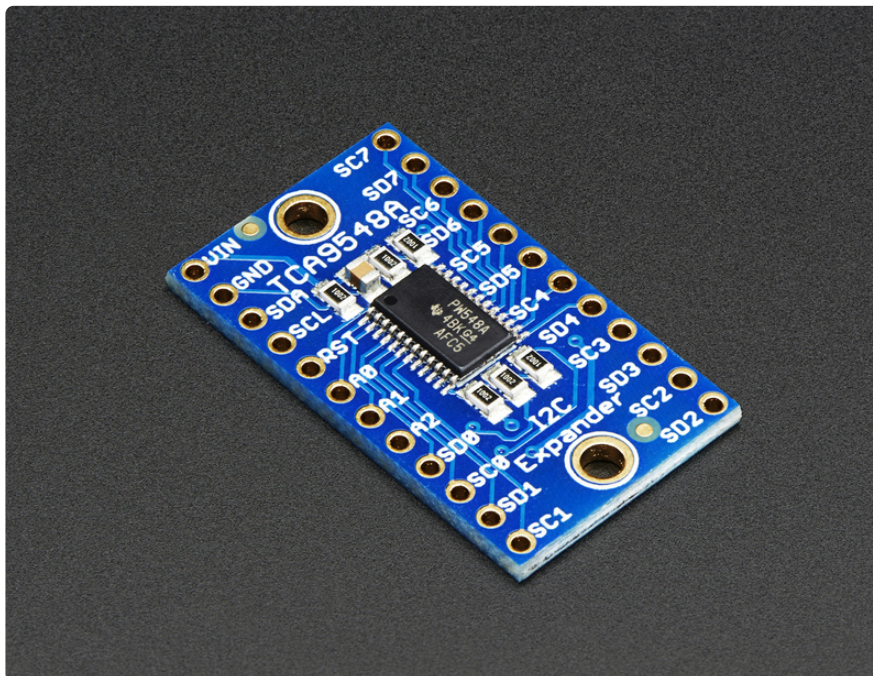


Adafruit PCA9548 8-Channel STEMMA
QT / Qwiic I2C Multiplexer

By Liz Clark

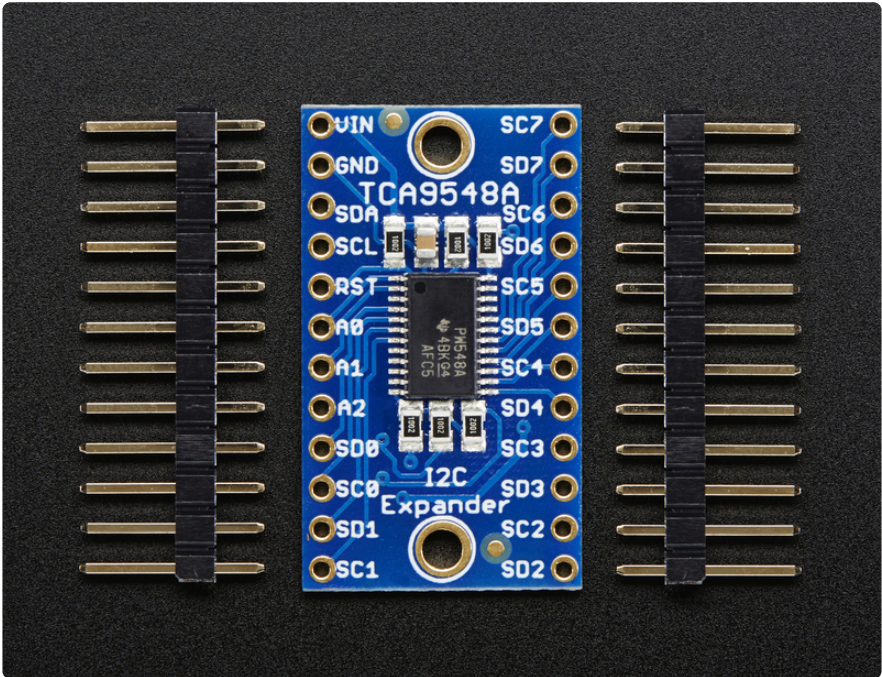
[Overview](#)

<https://learn.adafruit.com/adafruit-pca9548-8-channel-stemma-qt-qwiic-i2c-multiplexer/overview>

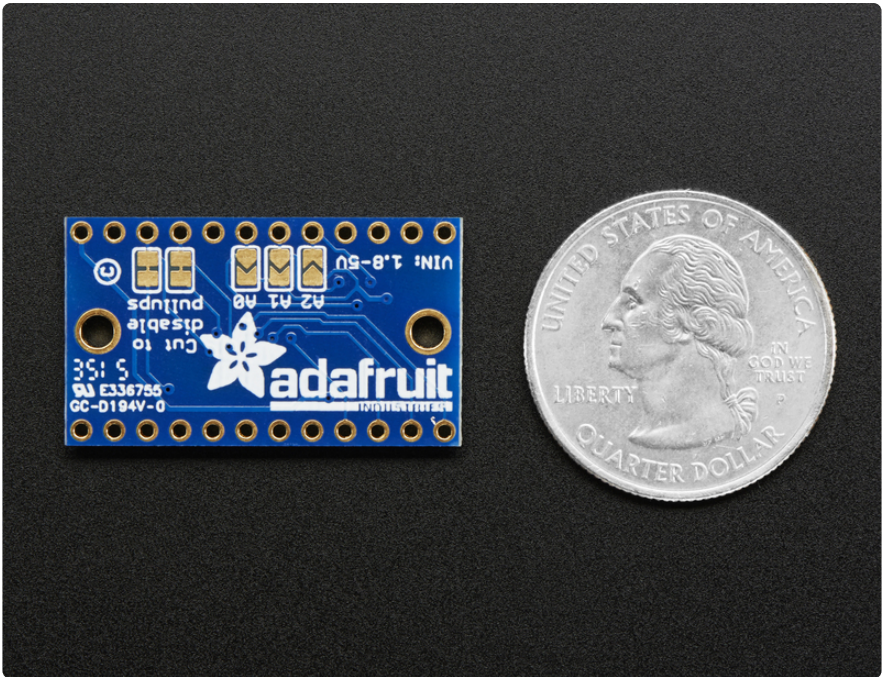


You just found the perfect I2C sensor, and you want to wire up two or three or more of them to your Arduino when you realize "Uh oh, this chip has a fixed I2C address, and from what I know about I2C, you cannot have two devices with the same address on

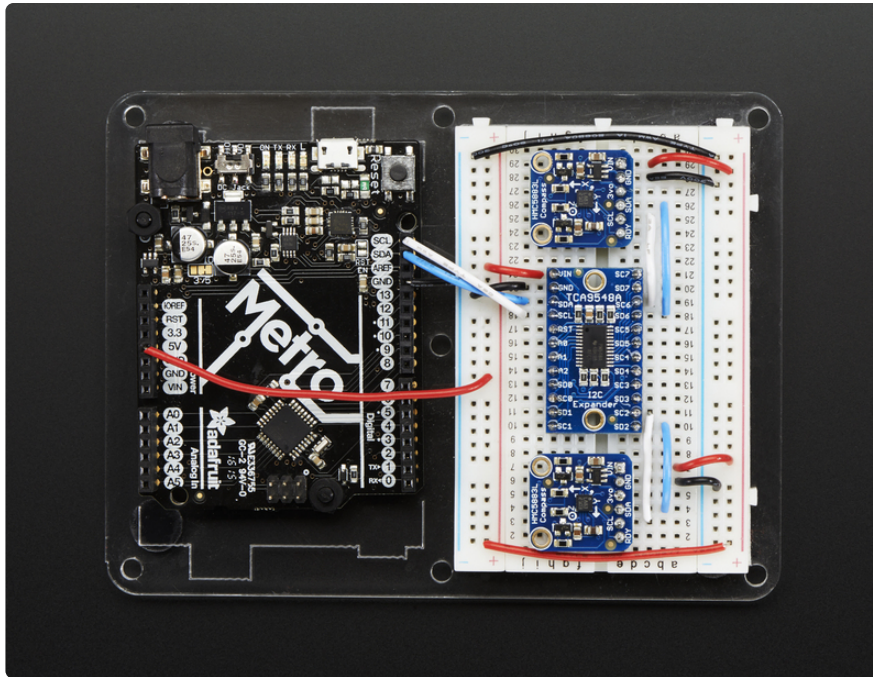
the same SDA/SCL pins!" Are you out of luck? You would be, if you didn't have this ultra-cool **TCA9548A 1-to-8 I2C multiplexer**!



Finally, a way to get up to 8 same-address I2C devices hooked up to one microcontroller - this multiplexer acts as a gatekeeper, shuttling the commands to the selected set of I2C pins with your command.

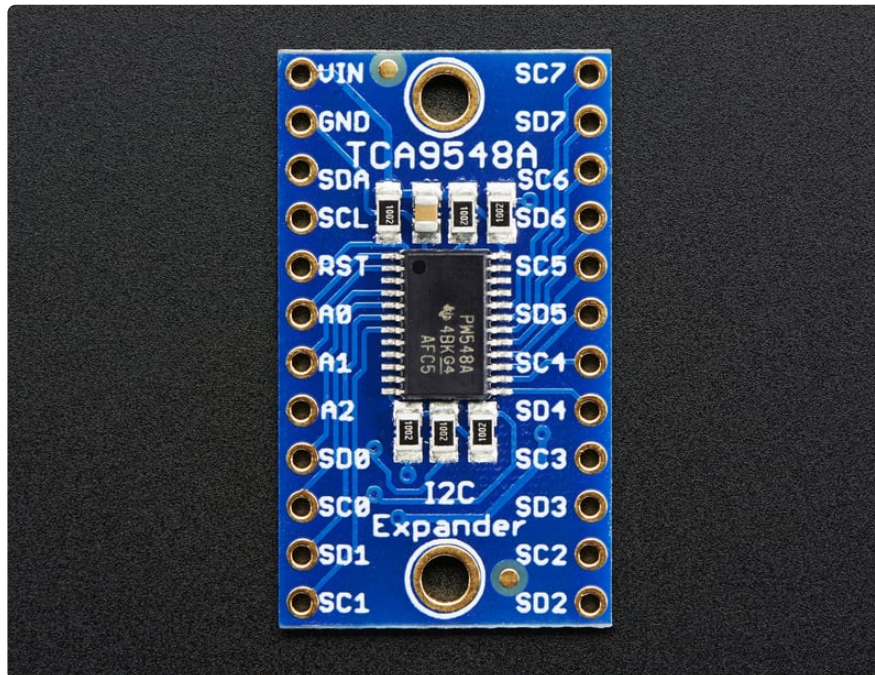


Using it is fairly straight-forward: the multiplexer itself is on I2C address 0x70 (but can be adjusted from 0x70 to 0x77) and you simply write a single byte with the desired multiplexed output number to that port, and bam - any future I2C packets will get sent to that port. In theory, you could have 8 of these multiplexers on each of 0x70-0x77 addresses in order to control 64 of the same-I2C-addressed-part.



Like all Adafruit breakouts, we put this nice chip on a breakout for you so you can use it on a breadboard with capacitors, and pullups and pulldowns to make usage a snap. Some header is required and once soldered in you can plug it into a solderless-breadboard. The chip itself is 1.8V - 5V compliant so you can use it with any logic level.

Pinouts



Power Pins:

- **Vin** - this is the power pin. Since the sensor chip uses 3-5 VDC. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V
- **GND** - common ground for power and logic

I2C Control-Side pins:

- **SCL** - this is the I2C clock pin for the chip itself, connect to your microcontrollers I2C clock line.
- **SDA** - this is the I2C data pin for the chip itself, connect to your microcontrollers I2C data line.
- **RST** - this is the reset pin, for resetting the multiplexer chip. Pulled high by default, connect to ground to reset.

I2C Address Selection

The default I2C address is **0x70**.

- **A0 A1 A2** - these are the address selection pins for the multiplexer. By default the multiplexer is at address **0x70** and these three pins are pulled low. Connect them to **Vin** to set the address to **0x71 - 0x77**.
- **A0** is the lowest-significant bit (if it is pulled high, it will increase the address by 1).
- **A1** is the 2nd-lowest-significant bit (if it is pulled high, it will increase the address by 2).
- **A2** is the 3rd-lowest-significant bit (if it is pulled high, it will increase the address by 4).

The default I2C address is **0x70**. The other address options can be calculated by “adding” the **A0/A1/A2** to the base of **0x70**.

A0 sets the lowest bit with a value of **1**, **A1** sets the next bit with a value of **2** and **A2** sets the next bit with a value of **4**. The final address is **0x70 + A2 + A1 + A0** which would be **0x77**.

So for example if **A2** is wired to power and **A0** is soldered closed, the address is **0x70 + 4 + 1 = 0x75**.

If only **A0** is wired to power, the address is **0x70 + 1 = 0x71**

If only **A1** is wired to power, the address is **0x70 + 2 = 0x72**

If only **A2** is wired to power, the address is **0x70 + 4 = 0x74**

The table below shows all possible addresses, and whether the pin(s) should be high/ H (wired to power) or low/L (unconnected).

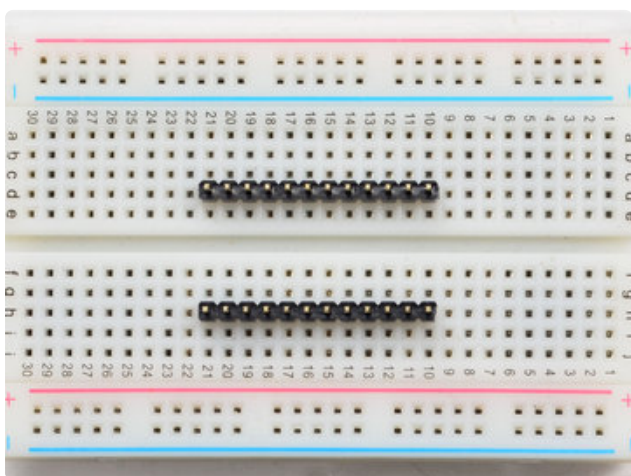
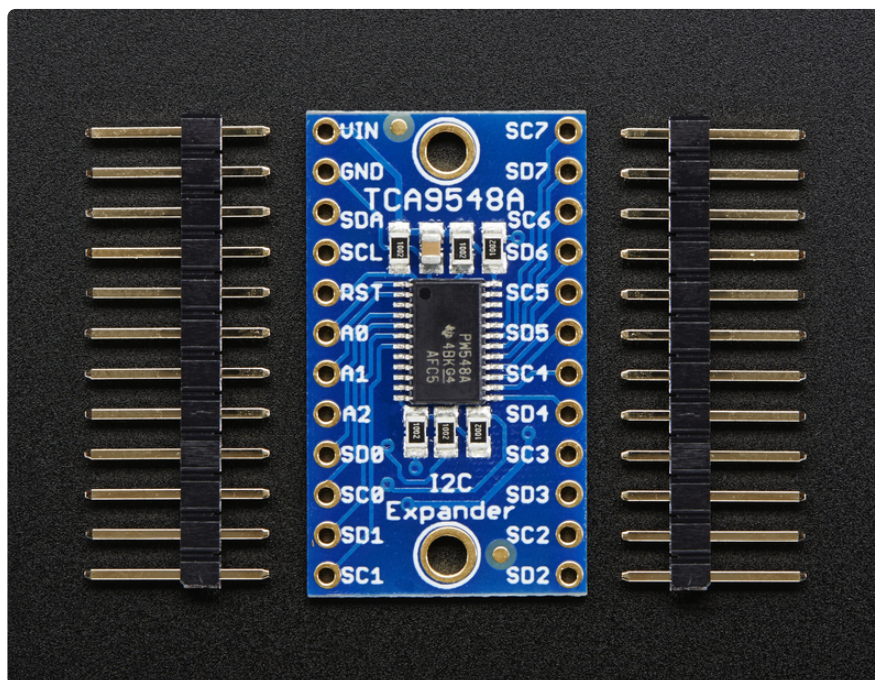
ADDR	A0	A1	A2	ADDR	A0	A1	A2
0x70	L	L	L	0x74	L	L	H
0x71	H	L	L	0x75	H	L	H
0x72	L	H	L	0x76	L	H	H
0x73	H	H	L	0x77	H	H	H

I2C Multiplexed-Side pins:

- **SDx** and **SCx**: There are 8 sets of **SDx** and **SCx** pins, from **SD0/SC0** to **SD7/SC7**. These are the multiplexed pins. Each one is a completely separate I2C bus set. So you can have 8 I2C devices with identical addresses, as long as they are on one I2C bus each.

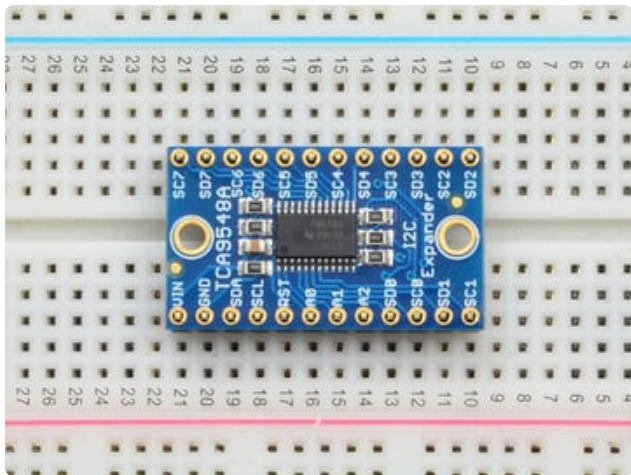
These pins do not have any pullups installed, so if you are using a chip or breakout without i2c pullups be sure to add them! Nicely, you can have **Vin** be 3.3V and have these pins pulled up to 5V (that is, they are 5V compliant)

Assembly



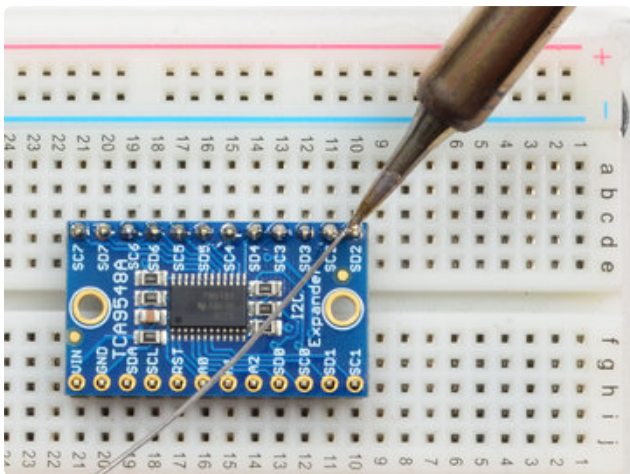
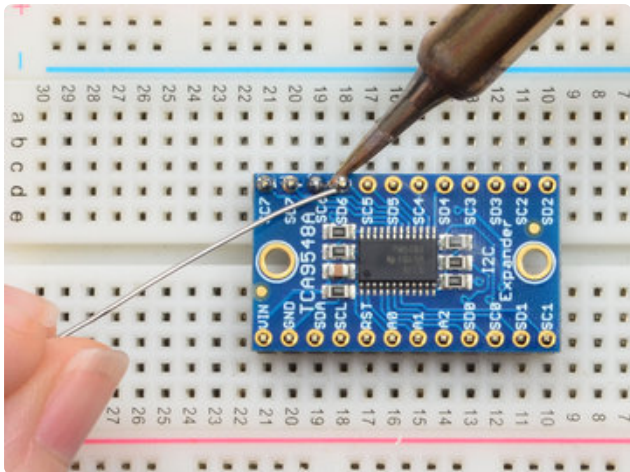
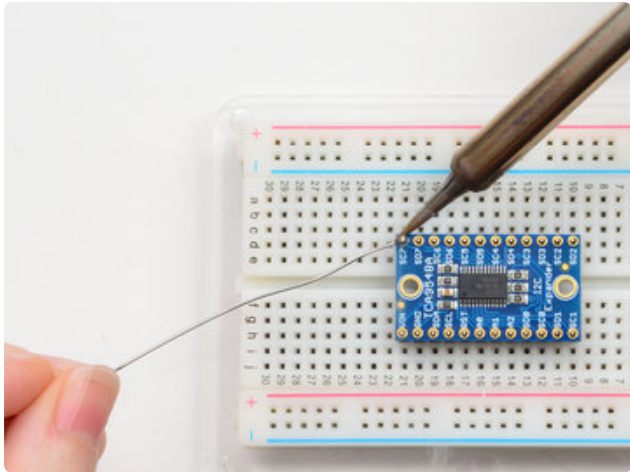
Prepare the header strip:

Cut the strip to length if necessary. It will be easier to solder if you insert it into a breadboard - **long pins down**



Add the breakout board:

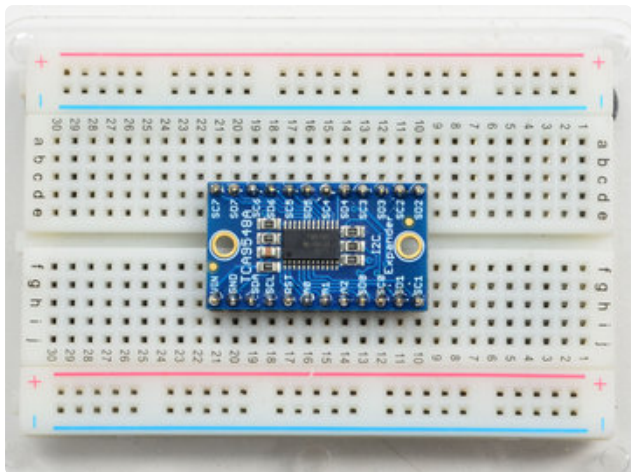
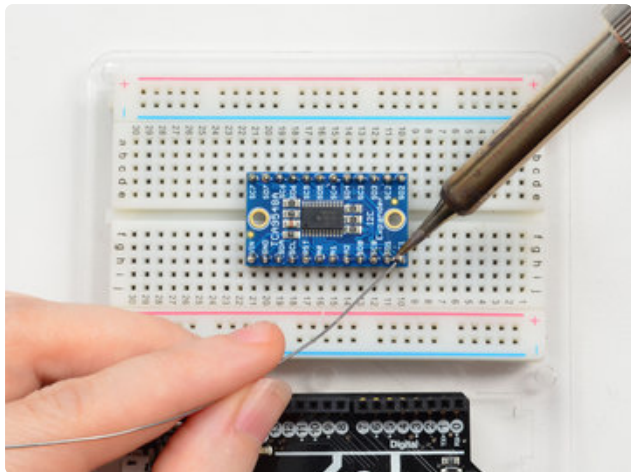
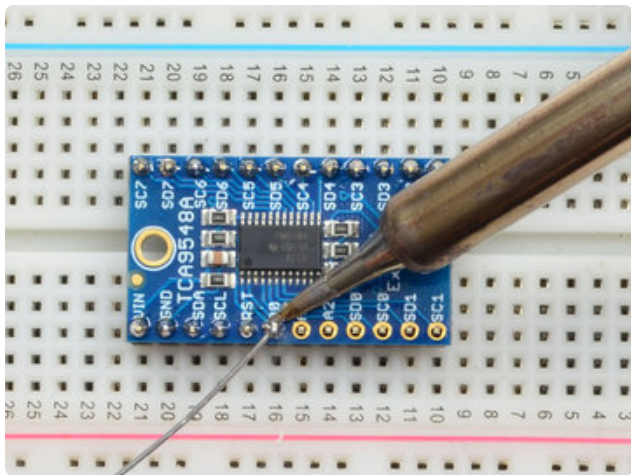
Place the breakout board over the pins so that the short pins poke through the breakout pads



And Solder!

Be sure to solder all pins for reliable electrical contact.

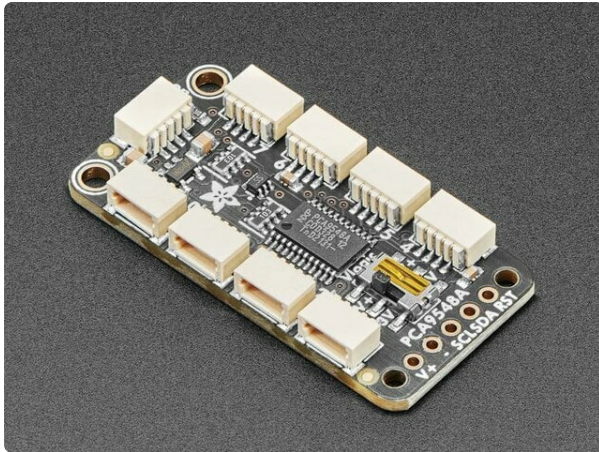
(For tips on soldering, be sure to check out our [Guide to Excellent Soldering](https://adafruit.it/aTk) (<https://adafruit.it/aTk>)).



You're done! Check your solder joints visually and continue onto the next steps

Arduino Wiring & Test

Also see [the guide \(https://adafru.it/11AB\)](https://adafru.it/11AB) on the Adafruit PCA9548 which is TCA9548A compatible and has Stemma QT ports for I2C.



Adafruit PCA9548 8-Channel STEMMA QT / Qwiic I2C Multiplexer

By Liz Clark

▯ [Arduino](#)

<https://learn.adafruit.com/adafruit-pca9548-8-channel-stemma-qt-qwiic-i2c-multiplexer/arduino>

The TCA9548A multiplexer is interesting in that it has an I2C address (0x70 by default) - and you basically send it a command to tell it which I2C multiplexed output you want to talk to, then you can address the board you want to address.

We suggest using this little helper to help you select the port

```
1  #define TCAADDR 0x70
2
3  void tcselect(uint8_t i) {
4      if (i > 7) return;
5
6      Wire.beginTransmission(TCAADDR);
7      Wire.write(1 << i);
8      Wire.endTransmission();
9  }
```

You can then call `tcselect(0)` thru `tcselect(7)` to set up the multiplexer.

Note that you if you happen to have I2C devices with I2C address 0x70, you will need to short one of the **Addr** pins on the TCA9548 breakout to **Vin** in order to make it not conflict. Given that you can have 0x70 thru 0x77, just find one that's free and you're good to go!

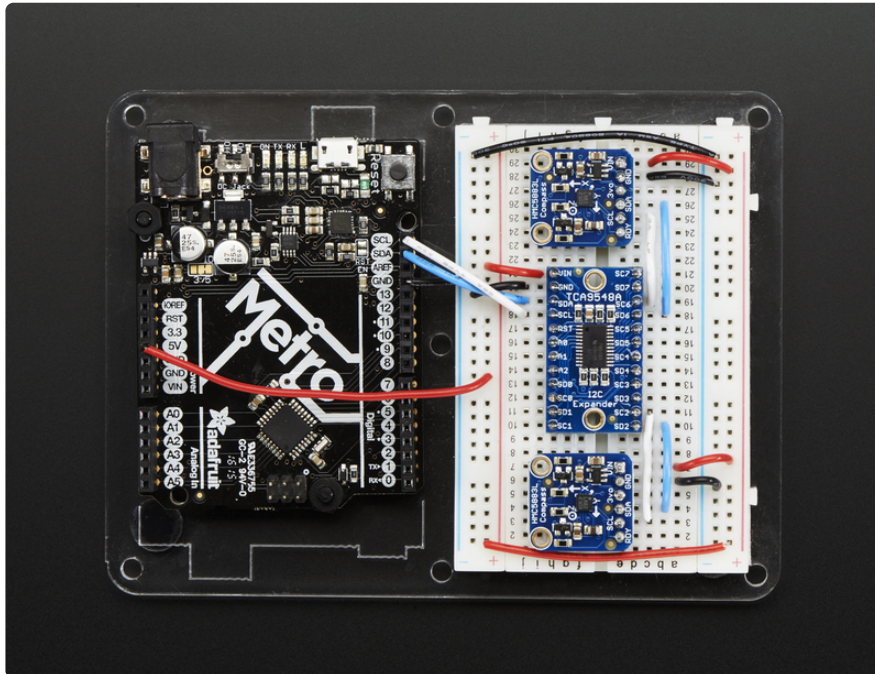
Example Multiplexing

For example, say we want to talk to two HMC5883 breakouts. These magnetometers have a fixed address of **0x1E** so you cannot have two on one I2C bus. Wire up the TCA9548 breakout so that:

- **Vin** is connected to **5V** (on a 3V logic Arduino/microcontroller, use **3.3V**)
- **GND** to ground
- **SCL** to I2C clock

- SDA to I2C data

Then wire up each of the other sensor breakouts to **Vin**, **Ground** and use one of the **SCn** / **SDn** multiplexed buses:



On an Arduino, which is what we're using, we suggest running this handy scanner script which will tell you what the breakout detected

```

1  /**
2   * TCA9548 I2CScanner.ino -- I2C bus scanner for Arduino
3   *
4   * Based on https://playground.arduino.cc/Main/I2cScanner/
5   *
6   */
7
8  #include "Wire.h"
9
10 #define TCAADDR 0x70
11
12 void tcaselect(uint8_t i) {
13   if (i > 7) return;
14
15   Wire.beginTransmission(TCAADDR);
16   Wire.write(1 << i);
17   Wire.endTransmission();
18 }
19
20 // standard Arduino setup()
21 void setup()

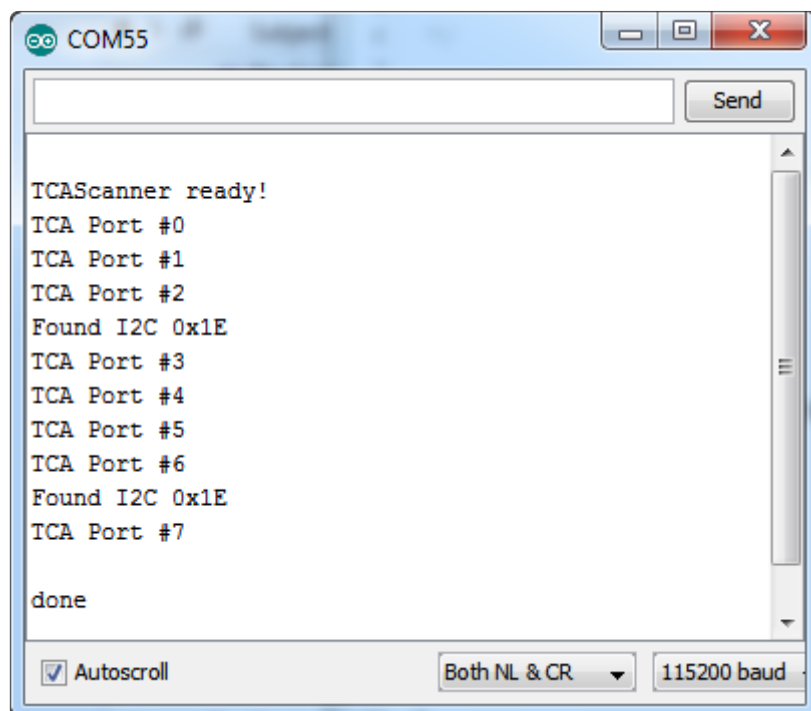
```

```

23 {
24   while (!Serial);
25   delay(1000);
26
27   Wire.begin();
28
29   Serial.begin(115200);
30   Serial.println("\nTCAScanner ready!");
31
32   for (uint8_t t=0; t<8; t++) {
33     tcselect(t);
34     Serial.print("TCA Port #"); Serial.println(t);
35
36     for (uint8_t addr = 0; addr<=127; addr++) {
37       if (addr == TCAADDR) continue;
38
39       Wire.beginTransmission(addr);
40       if (!Wire.endTransmission()) {
41         Serial.print("Found I2C 0x"); Serial.println(addr, HEX);
42       }
43     }
44   }
45   Serial.println("\ndone");
46 }
47
48 void loop()
49 {
50 }

```

For example, running it on the above setup will give you:



Next up you will have to adjust whatever code you have to select the correct multiplexed port!

Make sure before you query from the sensor that you call **tcselect** to get the right one

```
1  #include <Wire.h>
2  #include <Adafruit_Sensor.h>
3  #include <Adafruit_HMC5883_U.h>
4
5  #define TCAADDR 0x70
6
7  /* Assign a unique ID to this sensor at the same time */
8  Adafruit_HMC5883_Unified mag1 = Adafruit_HMC5883_Unified(1);
9  Adafruit_HMC5883_Unified mag2 = Adafruit_HMC5883_Unified(2);
10
11 void displaySensorDetails(Adafruit_HMC5883_Unified *mag)
12 {
13     sensor_t sensor;
14     mag->getSensor(&sensor);
15     Serial.println("-----");
16     Serial.print ("Sensor:   "); Serial.println(sensor.name);
17     Serial.print ("Driver Ver: "); Serial.println(sensor.version);
18     Serial.print ("Unique ID: "); Serial.println(sensor.sensor_id);
19     Serial.print ("Max Value: "); Serial.print(sensor.max_value); Serial.println(" uT");
20     Serial.print ("Min Value: "); Serial.print(sensor.min_value); Serial.println(" uT");
21     Serial.print ("Resolution: "); Serial.print(sensor.resolution); Serial.println(" uT");
22     Serial.println("-----");
23     Serial.println("");
24     delay(500);
25 }
26
27 void tcselect(uint8_t i) {
28     if (i > 7) return;
29
30     Wire.beginTransmission(TCAADDR);
31     Wire.write(1 << i);
32     Wire.endTransmission();
33 }
34
35
36 void setup(void)
37 {
38     Serial.begin(9600);
39     Serial.println("HMC5883 Magnetometer Test"); Serial.println("");
40
41     /* Initialise the 1st sensor */
42     tcselect(2);
43     if(!mag1.begin())
44     {
45         /* There was a problem detecting the HMC5883 ... check your connections */
46         Serial.println("Ooops, no HMC5883 detected ... Check your wiring!");
47         while(1);
48     }
49 }
```

```

50  /* Initialise the 2nd sensor */
51  tcselect(6);
52  if(!mag2.begin())
53  {
54      /* There was a problem detecting the HMC5883 ... check your connections */
55      Serial.println("Ooops, no HMC5883 detected ... Check your wiring!");
56      while(1);
57  }
58
59  /* Display some basic information on this sensor */
60  tcselect(2);
61  displaySensorDetails(&mag1);
62  tcselect(6);
63  displaySensorDetails(&mag2);
64  }
65
66  void loop(void)
67  {
68      /* Get a new sensor event */
69      sensors_event_t event;
70
71      tcselect(2);
72      mag1.getEvent(&event);
73
74      /* Display the results (magnetic vector values are in micro-Tesla (uT)) */
75      Serial.print("Sensor #1 - ");
76      Serial.print("X: "); Serial.print(event.magnetic.x); Serial.print(" ");
77      Serial.print("Y: "); Serial.print(event.magnetic.y); Serial.print(" ");
78      Serial.print("Z: "); Serial.print(event.magnetic.z); Serial.print(" "); Serial.println("uT");
79
80      tcselect(6);
81      mag2.getEvent(&event);
82      /* Display the results (magnetic vector values are in micro-Tesla (uT)) */
83      Serial.print("Sensor #2 - ");
84      Serial.print("X: "); Serial.print(event.magnetic.x); Serial.print(" ");
85      Serial.print("Y: "); Serial.print(event.magnetic.y); Serial.print(" ");
86      Serial.print("Z: "); Serial.print(event.magnetic.z); Serial.print(" "); Serial.println("uT");
87
88      delay(500);
89  }

```

However, once you add all the **tcselect()**'s you will be able to talk to both sensors!

Multiple Multiplexers

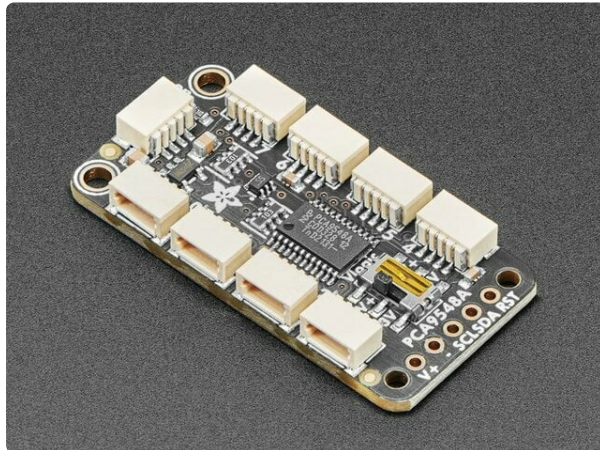
Since the TCA9548 is addressible, you can have more than one multiplexer on the bus. With 8 possible addresses, that means you can control as many as 64 separate i2c buses.

To avoid conflict between devices with the same address on different multiplexers, you can disable all channels on a multiplexer with the following code:

```
1 Wire.beginTransmission(TCAADDR1);  
2 Wire.write(0); // no channel selected  
3 Wire.endTransmission();
```

CircuitPython & Python

Also see [the guide \(https://adafru.it/19YD\)](https://adafru.it/19YD) on the Adafruit PCA9548 which is TCA9548A compatible and has Stemma QT ports for I2C.



Adafruit PCA9548 8-Channel STEMMA
QT / Qwiic I2C Multiplexer

By Liz Clark

▯ [CircuitPython & Python](#)

<https://learn.adafruit.com/adafruit-pca9548-8-channel-stemma-qt-qwiic-i2c-multiplexer/circuitpython-python>

It's easy to use the **TCA9548A** with Python or CircuitPython, and the [Adafruit_CircuitPython_TCA9548A \(https://adafru.it/ZYD\)](https://adafru.it/ZYD) module. This module allows you to easily write Python code that allows you to multiplex up to 8 STEMMA boards with the **TCA9548A** I2C multiplexer. You can use this multiplexer with any CircuitPython microcontroller board or with a computer that has GPIO and Python [thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library \(https://adafru.it/BSN\)](https://adafru.it/BSN).

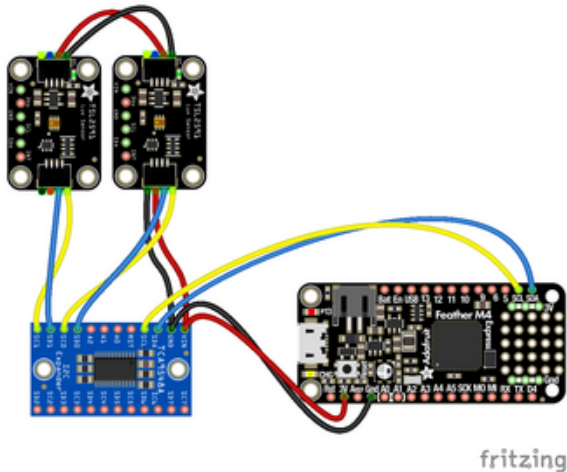
If you're curious why you'd need an I2C multiplexer, be sure to [check out this guide \(https://adafru.it/-af\)](https://adafru.it/-af) that goes in depth on working with **multiple** copies of the same I2C device, which most likely have the **same I2C address**.

<https://adafru.it/-af>

CircuitPython Microcontroller Wiring

First, wire up a TCA9548A to your board exactly as shown below. Here's an example of wiring a Feather M4 to the TCA9548A with I2C using one of the handy [STEMMA](#)

[QT \(https://adafru.it/Ft4\)](https://adafru.it/Ft4) connectors. Then, plug two TSL2591 STEMMA boards into the TCA9548A via STEMMA plug 0 and STEMMA plug 1:

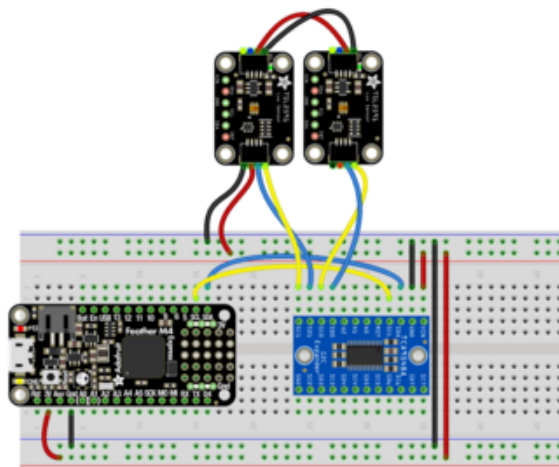


Board 3V to mux VIN (red wire)
Board GND to mux GND (black wire)
Board SCL to mux SCL (yellow wire)
Board SDA to mux SDA (blue wire)

TSL2591 1 SDA to mux SD0
TSL2591 1 SCL to mux SC0
TSL2591 1 VIN to mux VIN
TSL2591 1 GND to mux GND

TSL2591 2 SDA to mux SD1
TSL2591 2 SDA to mux SC1
TSL2591 2 VIN to mux VIN
TSL2591 2 GND to mux GND

You can also use standard **0.100"** pitch headers to wire it up on a breadboard:



Board 3V to mux VIN (red wire)
 Board GND to mux GND (black wire)
 Board SCL to mux SCL (yellow wire)
 Board SDA to mux SDA (blue wire)

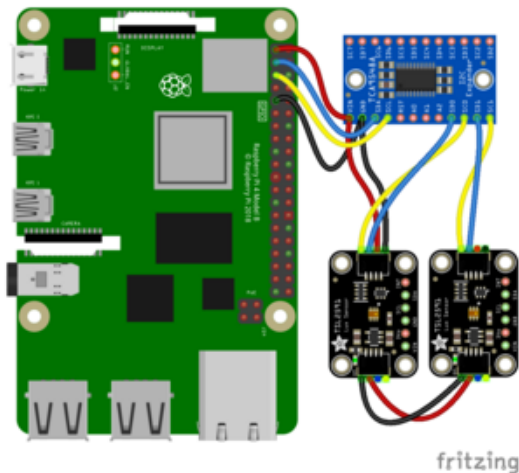
TSL2591 1 SDA to mux SD0
 TSL2591 1 SCL to mux SC0
 TSL2591 1 VIN to mux VIN
 TSL2591 1 GND to mux GND

TSL2591 2 SDA to mux SD1
 TSL2591 2 SCL to mux SC1
 TSL2591 2 VIN to mux VIN
 TSL2591 2 GND to mux GND

Python Computer Wiring

Since there's dozens of Linux computers/boards you can use, below shows wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported](https://adafruit.it/BSN) (<https://adafruit.it/BSN>).

Here's the Raspberry Pi wired to the I2C multiplexer using I2C and a [STEMMA QT](https://adafruit.it/Ft4) (<https://adafruit.it/Ft4>) connector:

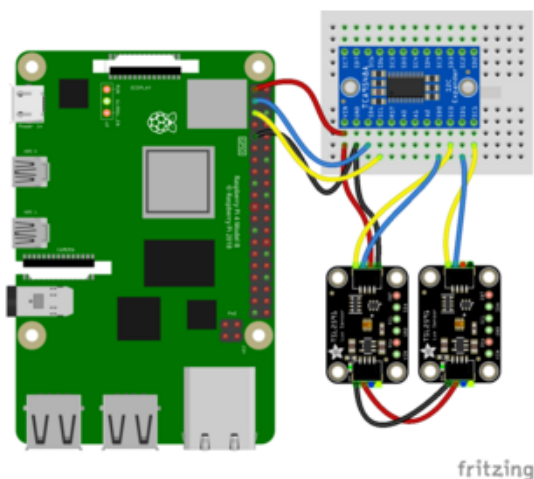


Pi 3V to mux VIN (red wire)
 Pi GND to mux GND (black wire)
 Pi SCL to mux SCL (yellow wire)
 Pi SDA to mux SDA (blue wire)

TSL2591 1 SDA to mux SD0
 TSL2591 1 SCL to mux SC0
 TSL2591 1 VIN to mux VIN
 TSL2591 1 GND to mux GND

TSL2591 2 SDA to mux SD1
 TSL2591 2 SDA to mux SC1
 TSL2591 2 VIN to mux VIN
 TSL2591 2 GND to mux GND

Finally here is an example of how to wire up a Raspberry Pi to the I2C multiplexer using a solderless breadboard:



Pi 3V to mux VIN (red wire)
 Pi GND to mux GND (black wire)
 Pi SCL to mux SCL (yellow wire)
 Pi SDA to mux SDA (blue wire)

TSL2591 1 SDA to mux SD0
 TSL2591 1 SCL to mux SC0
 TSL2591 1 VIN to mux VIN
 TSL2591 1 GND to mux GND

TSL2591 2 SDA to mux SD1
 TSL2591 2 SDA to mux SC1
 TSL2591 2 VIN to mux VIN
 TSL2591 2 GND to mux GND

Python Installation of TCA9548A Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. [Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready \(https://adafru.it/BSN\)](https://adafru.it/BSN)!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-tca9548a`

If your default Python is version 3, you may need to run `pip` instead. Make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

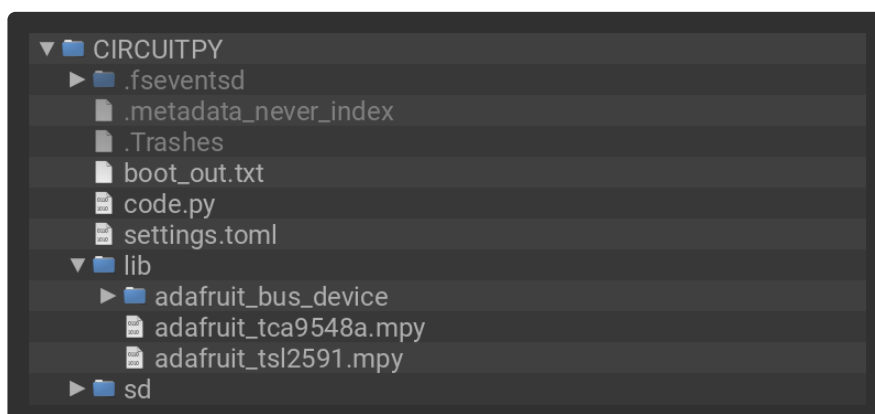
CircuitPython Usage

To use with CircuitPython, you need to first install the TCA9548A library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folders and file:

- **adafruit_bus_device/**
- **adafruit_tca9548a.mpy**
- **adafruit_tsl2591.mpy**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing `code.py` with whatever you named the file:

```
python3 code.py
```

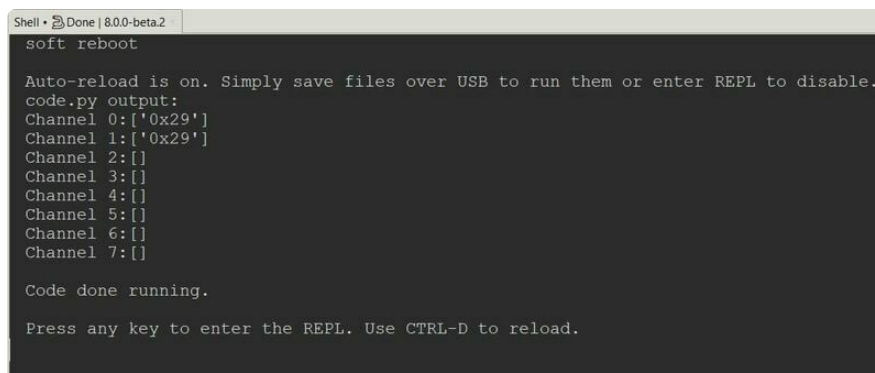
Simple Test Example Code

```
1 # SPDX-FileCopyrightText: 2021 Carter Nelson for Adafruit Industries
2 # SPDX-License-Identifier: MIT
3
4 # This example shows using TCA9548A to perform a simple scan for connected devices
5 import board
6
7 import adafruit_tca9548a
8
9 # Create I2C bus as normal
10 i2c = board.I2C() # uses board.SCL and board.SDA
11 # i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a microcontroller
12
13 # Create the TCA9548A object and give it the I2C bus
14 tca = adafruit_tca9548a.TCA9548A(i2c)
15
16 for channel in range(8):
17     if tca[channel].try_lock():
18         print(f"Channel {channel}: ", end="")
19         addresses = tca[channel].scan()
20         print([hex(address) for address in addresses if address != 0x70])
21     tca[channel].unlock()
```

https://github.com/adafruit/Adafruit_CircuitPython_TCA9548A/blob/main/examples/tca9548a_simpletest.py

If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, [connect to the serial console](#) (<https://adafru.it/Bec>) to see the data printed out!

If running Python: The console output will appear wherever you are running Python.



```
Shell • Done | 8.0.0-beta.2
soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
Channel 0: ['0x29']
Channel 1: ['0x29']
Channel 2: []
Channel 3: []
Channel 4: []
Channel 5: []
Channel 6: []
Channel 7: []

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.
```

In this simple test for the TCA9548A, an I2C scan is performed for all eight of its ports. If any devices are connected, then the I2C address will be printed to the REPL next to the channel number. If no device is connected, then the port will print with empty brackets (`[]`).

Multi-Sensor Example Code

```
1  # SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
2  # SPDX-License-Identifier: MIT
3
4  # This example shows using two TSL2491 light sensors attached to TCA9548A channels 0 and 1.
5  # Use with other I2C sensors would be similar.
6  import time
7
8  import adafruit_tsl2591
9  import board
10
11  import adafruit_tca9548a
12
13  # Create I2C bus as normal
14  i2c = board.I2C() # uses board.SCL and board.SDA
15  # i2c = board.STEMMA_I2C() # For using the built-in STEMMA QT connector on a microcontroller
16
17  # Create the TCA9548A object and give it the I2C bus
18  tca = adafruit_tca9548a.TCA9548A(i2c)
19
20  # For each sensor, create it using the TCA9548A channel instead of the I2C object
21  tsl1 = adafruit_tsl2591.TSL2591(tca[0])
22  tsl2 = adafruit_tsl2591.TSL2591(tca[1])
23
24  # After initial setup, can just use sensors as normal.
25  while True:
26      print(tsl1.lux, tsl2.lux)
27      time.sleep(0.1)
```

https://github.com/adafruit/Adafruit_CircuitPython_TCA9548A/blob/main/examples/tca9548a_multisensor.py

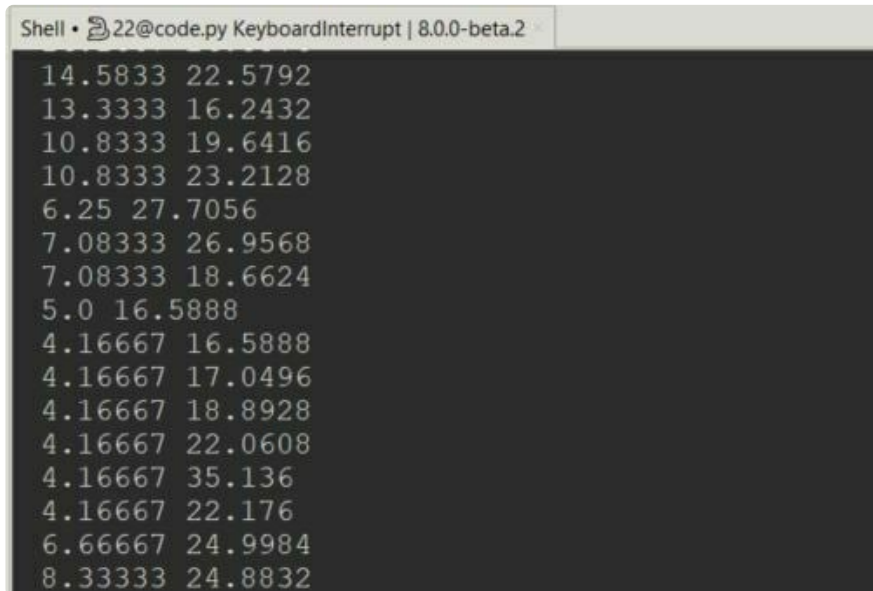
In the multi-sensor example, the TCA9548A is used as an I2C multiplexer with two TSL2591 light sensors. When the connected sensors are instantiated over I2C, the I2C pins declared are the ports from `0` and `1` on the TCA9548A.

In the example, the first TSL2591 light sensor, instantiated as `tsl1`, is plugged into port 0 (`tca[0]`) and the second TSL2591 light sensor, instantiated as `tsl2`, is plugged into port 1 (`tca[1]`).

```
1  # Create the TCA9548A object and give it the I2C bus
2  tca = adafruit_tca9548a.TCA9548A(i2c)
3
4  # For each sensor, create it using the TCA9548A channel instead of the I2C object
5  tsl1 = adafruit_tsl2591.TSL2591(tca[0])
6  tsl2 = adafruit_tsl2591.TSL2591(tca[1])
```

In the loop, the readings from the two light sensors are printed to the REPL every **0.1** seconds.

```
1 while True:
2     print(tsl1.lux, tsl2.lux)
3     time.sleep(0.1)
```

A screenshot of a terminal window titled "Shell • 22@code.py KeyboardInterrupt | 8.0.0-beta.2". The terminal displays a series of two columns of floating-point numbers, representing light sensor readings. The values range from approximately 4.16667 to 14.58333 in the first column, and 16.2432 to 27.7056 in the second column. The readings are printed every 0.1 seconds as indicated by the code above.

tsl1.lux	tsl2.lux
14.5833	22.5792
13.3333	16.2432
10.8333	19.6416
10.8333	23.2128
6.25	27.7056
7.08333	26.9568
7.08333	18.6624
5.0	16.5888
4.16667	16.5888
4.16667	17.0496
4.16667	18.8928
4.16667	22.0608
4.16667	35.136
4.16667	22.176
6.66667	24.9984
8.33333	24.8832

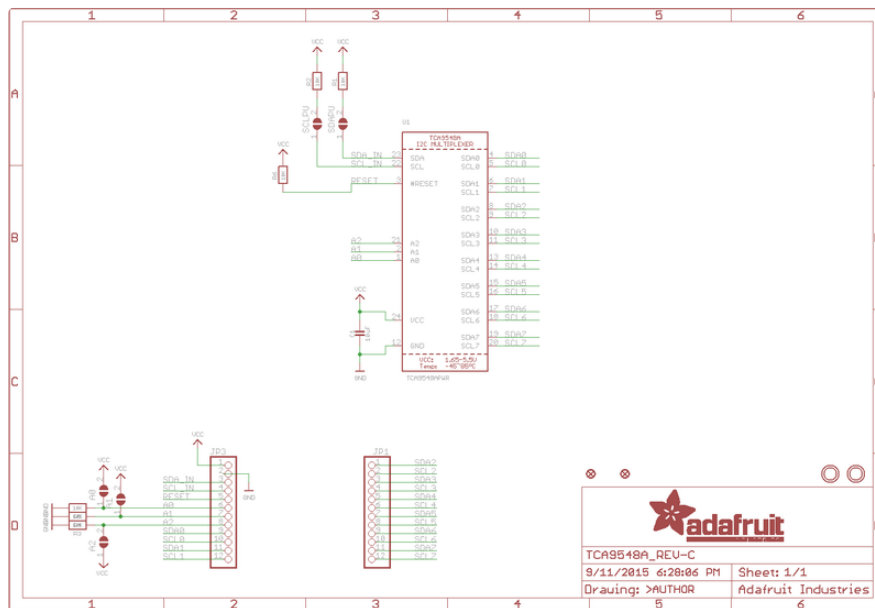
Downloads

Datasheets

- [TCA9548A datasheet \(https://adafru.it/id8\)](https://adafru.it/id8)
- [Fritzing object in the Adafruit Fritzing Library \(https://adafru.it/aP3\)](https://adafru.it/aP3)
- [EagleCAD PCB files on GitHub \(https://adafru.it/rzF\)](https://adafru.it/rzF)

Schematic

Click to embiggen



Fabrication Print

Dimensions in Inches

